GNN AND RL-BASED ATPG FOR OPEN-SOURCE RISC-V PROCESSOR CORES

Project – I (2026-2027)

Members:

Somesh TG -23BLC1023

Shreya Ranjitha – 23BLC1039

Mridula B – 23BLC1043

Guide: 11/8/26

Dr. Ananiah Durai

School of Electronics Engineering

GNN AND RL-BASED ATPG FOR OPEN-SOURCE RISC-V PROCESSOR CORES

Project – I (2026-2027)

Members:

Somesh TG -23BLC1023

Shreya Ranjitha – 23BLC1039

Mridula B – 23BLC1043

Guide:

Dr. Ananiah Durai

School of Electronics Engineering

INTRODUCTION

Automatic Test Pattern Generation (ATPG) is a key step in semiconductor testing that automatically generates input test patterns to detect manufacturing faults such as stuck-at-0, stuck-at-1, and transition faults. Conventional ATPG techniques, including PODEM, FAN, and SAT-based approaches, rely heavily on heuristic search and backtracking, which can become computationally expensive as circuit complexity increases. Recent research has explored Machine Learning (ML), particularly Graph Neural Networks (GNNs) and Reinforcement Learning (RL), to improve ATPG search efficiency. However, most existing ML-based ATPG methods have been evaluated primarily on standard benchmark circuits such as ISCAS and ITC or on industrial designs. Open-source RISC-V processor cores provide realistic, publicly available and reproducible processor-scale circuits, offering an opportunity to investigate the effectiveness and generalization of ML-based ATPG on complex processor architectures. The proposed work therefore explores GNN-based circuit representation combined with RL-guided ATPG search for open-source RISC-V cores.

PROBLEM STATEMENT

Problem

Traditional ATPG can experience:

- Large search spaces for difficult-to-detect faults
- Excessive backtracking
- High ATPG execution time
- Increasing test-pattern generation complexity for large circuits

Although recent ML-based ATPG methods have demonstrated improvements, there is a lack of systematic evaluation on open source processor-scale RISC-V cores.

Research Gap

- ML-ATPG research mainly targets ISCAS, ITC benchmark circuits and industrial designs.
- RISC-V + ML research focuses primarily on verification, reliability, security and physical design.
- Limited work combines ML-based ATPG with open-source RISC-V processor cores.
- Cross-core generalization of learned ATPG strategies remains largely unexplored.
- This work investigates GNN + RL-based ATPG on realistic, reproducible RISC-V processor netlists.

OBJECTIVES

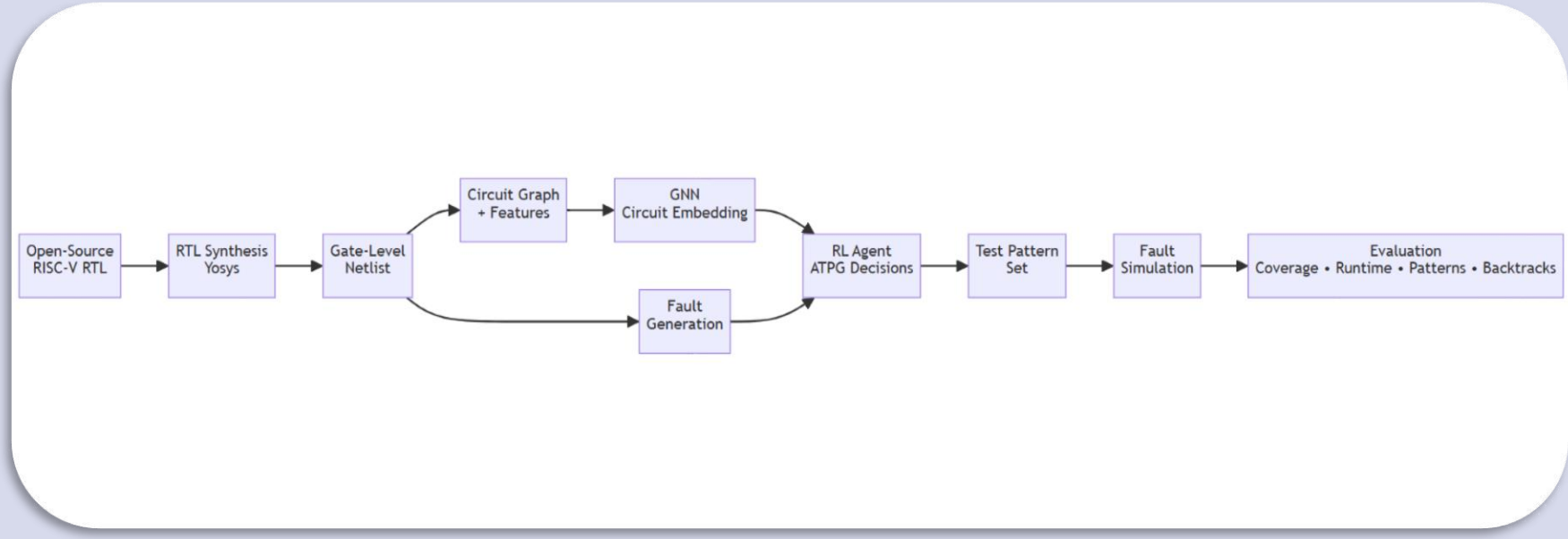
Primary Objectives:

Develop and evaluate an ML-guided ATPG framework for open-source RISC-V processor cores using GNN-based circuit representation and Reinforcement Learning-based search guidance.

Specific Objectives:

- Synthesize selected open-source RISC-V cores into gate-level netlists.
- Represent the synthesized circuit as a graph of logic gates and interconnections.
- Extract circuit features such as:
- Gate type
- Fanout
- Depth
- SCOAP controllability/observability
- Logic state
- Use a GNN to learn circuit-structural representations.
- Use Reinforcement Learning to guide ATPG decision making and reduce unnecessary backtracking.
- Compare ML-guided ATPG against conventional ATPG baselines.
- Evaluate fault coverage, pattern count, runtime and backtracking.
- Investigate cross-core generalization between different RISC-V implementations.

PROPOSED METHODOLOGY



Architecture Diagram

PROPOSED METHODOLOGY

Steps:

1. RISC-V Core Selection
- Ibex
- PicoRV32
- Potentially Rocket/CVA6 depending on computational feasibility

2. Logic Synthesis
- Convert RTL into technology-mapped gate-level netlists using Yosys.

3. Fault Modeling
- Generate stuck-at faults at relevant circuit nodes/gate pins.

4. Graph Construction
- Gates → graph nodes
- Wires → graph edges

5. GNN Representation
- Learn structural representations of circuit regions and fault-related states.

6. RL-guided ATPG
- The RL agent selects promising search/backtrace decisions.

7. Fault Simulation
- Evaluate generated patterns against the fault list.

8. Performance Evaluation
- Compare against conventional ATPG.

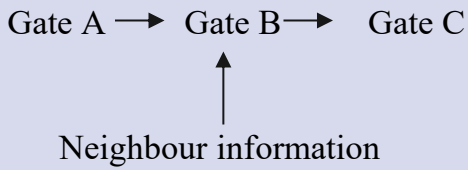
PROPOSED METHODOLOGY

GNN Component

Each circuit node contains features such as:

- Gate Type
- Logic Value
- Fanout
- Depth
- SCOAP Controllability
- SCOAP Observability
- Fault Information

The GNN performs message passing between connected gates:



Result:
Circuit embedding representing the structural and logical context of the ATPG state.

PROPOSED METHODOLOGY

Reinforcement Learning Component

State:
- Current ATPG state
- Fault information
- GNN embeddings

Action:
Select the next ATPG decision/search direction

Reward:
- Positive → successful fault detection
- Negative → conflict / unnecessary backtracking
- Penalty → excessive search

Goal
Learn ATPG decisions that reach detectable patterns with fewer unnecessary searches.

EXPECTED OUTCOME

Technical Outcomes

- Evaluate stuck-at fault coverage achieved by ML-guided ATPG across selected open-source RISC-V cores.
- Measure reduction in ATPG backtracking and search effort compared with conventional ATPG baselines such as PODEM.
- Evaluate ATPG efficiency using runtime, test-pattern count, and performance on hard-to-detect faults.
- Investigate cross-core generalisation by testing whether a model trained on one RISC-V core can be applied to another with limited performance degradation.
- Identify circuit characteristics and ML features that influence the effectiveness of GNN/RL-based ATPG.

Deliverables & Contributions

- A dataset of synthesised RISC-V gate-level netlists, fault lists, and circuit features for selected cores.
- A prototype ML-guided ATPG framework integrating circuit-graph representation, GNN-based encoding, and RL-based search guidance.
- A baseline comparison between conventional and ML-guided ATPG using fault coverage, runtime, pattern count, and backtracking.
- A cross-core generalisation study evaluating transferability of learned circuit representations.
- Research documentation and experimental results suitable for a project report and potential research publication.

TOOLS & TECHNOLOGIES

EDA & ATPG

Yosys - RTL synthesis and gate-level netlist generation
Fault - DFT, ATPG and fault simulation
Ibex / PicoRV32 - Open-source RISC-V cores

Machine Learning

Python
PyTorch
PyTorch Geometric
NumPy

Development & Analysis

Google Colab
Pandas
Matplotlib
Git / GitHub

PROJECT TIMELINE

Phase	Activities	Timeline
1. Literature, Dataset & Conventional ATPG	Literature survey; RISC-V core selection; synthesis; gate-level netlist generation; fault-list and circuit-feature dataset creation; run conventional ATPG baseline	Weeks 1–4
2. ML-ATPG Development	Circuit graph construction; feature extraction; GNN encoder; RL-based ATPG search; initial ML-ATPG experiments	Weeks 5–8
3. Evaluation & Generalisation	Compare conventional vs ML-ATPG using coverage, runtime, backtracks and pattern count; test on a second RISC-V core if feasible; analysis and final documentation	Weeks 9–12

REFERENCES

[1] W. Li, H. Lyu, S. Liang, T. Wang, et al., “SmartATPG: Learning-based Automatic Test Pattern Generation with Graph Convolutional Network and Reinforcement Learning,” *61st ACM/IEEE Design Automation Conference (DAC)*, 2024.

[2] B. Sun, R. Zhang, Z. Chao, Z. Liu, J. Mu, J. Ye, and H. Li, “InF-ATPG: Intelligent FFR-Driven ATPG with Advanced Circuit Representation Guided Reinforcement Learning,” 2025.

[3] Y. Ye, Z. Wang, Z. Xue, Z. Wang, Y. Gao, H. Yan, and L. Shi, “FPGNN-ATPG: An Efficient Fault Parallel Automatic Test Pattern Generator,” *Great Lakes Symposium on VLSI (GLSVLSI)*, pp. 299–304, 2023.

[4] W. Li, H. Lyu, S. Liang, T. Wang, et al., “Intelligent Automatic Test Pattern Generation for Digital Circuits Based on Reinforcement Learning,” *IEEE Asian Test Symposium (ATS)*, 2023.

[5] S. Roy, S. K. Millican, and V. D. Agrawal, “Machine Intelligence for Efficient Test Pattern Generation,” *IEEE International Test Conference (ITC)*, pp. 1–5, 2020.

[6] S. Roy, V. D. Agrawal, and S. K. Millican, “Training Neural Network for Machine Intelligence in Automatic Test Pattern Generator,” *IEEE VLSI Design and Test Symposium (VLSID)*, 2021.

[7] Z. Chao, B. Sun, H. Lyu, G. Yu, M. Wang, et al., “HighTPI: A Hierarchical Graph Based Intelligent Method for Test Point Insertion,” *IEEE VLSI Test Symposium (VTS)*, 2025.

[8] Z. Shi, M. Li, S. Khan, L. Wang, N. Wang, Y. Huang, and Q. Xu, “DeepTPI: Test Point Insertion with Deep Reinforcement Learning,” *IEEE International Test Conference (ITC)*, pp. 194–203, 2022.

[9] D. Mao, D. Yan, Z. Gao, Z. Liu, et al., “RDT: An Optimized Automatic Test Pattern Generation Method Based on Robust Decision Tree,” *International Symposium of Electronics Design Automation (ISEDA)*, 2025.

[10] V. I. Kuraedov, “Application of Machine Learning Methods for Automatic Test Pattern Generation Process Optimization During Circuit Design,” *Neurocomputers*, vol. 26, no. 1, pp. 14–22, 2024.

REFERENCES

[11] J. P. Roth, “Diagnosis of Automata Failures: A Calculus and a Method,” *IBM Journal of Research and Development*, vol. 10, no. 4, pp. 278–291, 1966.
Foundational D-algorithm reference.
[12] P. Goel, “An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits,” *IEEE Transactions on Computers*, vol. C-30, no. 3, pp. 215–222, 1981.
Foundational PODEM reference.
[13] H. Fujiwara and T. Shimono, “On the Acceleration of Test Generation Algorithms,” *IEEE Transactions on Computers*, vol. C-32, pp. 1137–1144, 1983.
Important FAN / backtracking reduction reference.
[14] L. H. Goldstein and E. L. Thigpen, “SCOAP: Sandia Controllability/Observability Analysis Program,” *Proceedings of the 17th Design Automation Conference*, pp. 190–196, 1980.
Important for your SCOAP features.
[15] V. D. Agrawal, “The Need for Fault Models in VLSI Testing,” *Proceedings of the IEEE*, relevant foundational work on fault modeling and test generation.
[16] L. Alrahis, J. Knechtel, F. Klemme, H. Amrouch, and O. Sinanoglu, “GNN4REL: Graph Neural Networks for Predicting Circuit Reliability Degradation,” 2022.
This one is particularly useful for your RISC-V angle, because the authors evaluate GNN-based reliability prediction on RISC-V processors in addition to EPFL and ITC-99 circuits.
[17] H.-G. Stratigopoulos, “Machine Learning Applications in IC Testing,” *IEEE European Test Symposium (ETS)*, 2018.
[18] “Graph Neural Networks for Integrated Circuit Design, Reliability, and Security: Survey and Tool,” *ACM Computing Surveys*, 2025.

REFERENCES

[11] J. P. Roth, “Diagnosis of Automata Failures: A Calculus and a Method,” *IBM Journal of Research and Development*, vol. 10, no. 4, pp. 278–291, 1966.
Foundational D-algorithm reference.
[12] P. Goel, “An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits,” *IEEE Transactions on Computers*, vol. C-30, no. 3, pp. 215–222, 1981.
Foundational PODEM reference.
[13] H. Fujiwara and T. Shimono, “On the Acceleration of Test Generation Algorithms,” *IEEE Transactions on Computers*, vol. C-32, pp. 1137–1144, 1983.
Important FAN / backtracking reduction reference.
[14] L. H. Goldstein and E. L. Thigpen, “SCOAP: Sandia Controllability/Observability Analysis Program,” *Proceedings of the 17th Design Automation Conference*, pp. 190–196, 1980.
Important for your SCOAP features.
[15] V. D. Agrawal, “The Need for Fault Models in VLSI Testing,” *Proceedings of the IEEE*, relevant foundational work on fault modeling and test generation.
[16] L. Alrahis, J. Knechtel, F. Klemme, H. Amrouch, and O. Sinanoglu, “GNN4REL: Graph Neural Networks for Predicting Circuit Reliability Degradation,” 2022.
This one is particularly useful for your RISC-V angle, because the authors evaluate GNN-based reliability prediction on RISC-V processors in addition to EPFL and ITC-99 circuits.
[17] H.-G. Stratigopoulos, “Machine Learning Applications in IC Testing,” *IEEE European Test Symposium (ETS)*, 2018.
[18] “Graph Neural Networks for Integrated Circuit Design, Reliability, and Security: Survey and Tool,” *ACM Computing Surveys*, 2025.

REFERENCES

[19] F. Zaruba and L. Benini, “The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 11, pp. 2629–2640, 2019.
Useful as the CVA6/Ariane processor reference.
[20] A. Waterman et al. / Berkeley architecture team, “The Rocket Chip Generator,” RISC-V processor-generator work.
Useful for establishing the use of open-source configurable RISC-V processor architectures; Rocket Chip generates a range of RISC-V-based SoCs.
[21] W. Li, Y. Zou, Y. Liang, J. Moura, and S. Blanton, “DEFT: Differentiable Automatic Test Pattern Generation,” 2025.
Very relevant recent work: uses differentiable optimization for ATPG, particularly hard-to-detect faults.
[22] L. Zhang and A. Davoodi, “Hybrid ATPG: Neural Network Assisted Automatic Test Pattern Generation,” 2024.
Particularly useful because the accompanying dataset/code compares neural-network-assisted ATPG with FAN and includes hard-to-detect fault experiments.
[23] “Reinforcement Learning Framework for Adaptive Test Pattern Generation in Logic and Mixed-Signal ICs,” 2022.
Relevant as another RL-based ATPG formulation.
[24] “SAT-based Automatic Test Pattern Generation,” R. Drechsler, S. Eggersglüß, G. Fey, and D. Tille, 2009.
Useful for discussing SAT-based alternatives to heuristic ATPG.
[25] S. C. Jiang, C. L. Lee, W. Z. Shen, J. E. Chen, and C. P. Wu, “LDS-ATPG: An Automatic Test Pattern Generation System for Combinational VLSI Circuits,” 1989.